UD1 – BOLETÍN 3 · EJERCICIO 3: TiendaTech (CSV + JSON + TXT)

En este ejercicio vas a construir el sistema de gestión de una pequeña tienda online de informática. A diferencia de los ejercicios anteriores, **la "base de datos" está repartida en tres ficheros de formatos distintos**, que tendrás que leer, relacionar entre sí, validar, modificar y volver a escribir:

Fichero	Formato	Contenido
`productos.csv`	CSV con cabecera	Catálogo de productos
`clientes.json`	JSON con objetos anidados y listas	Clientes de la tienda
`pedidos.txt`	Texto plano con formato propio	Histórico de pedidos

Además, el sistema **generará** ficheros de los tres tipos (log, facturas, informes y copias de seguridad).

Todos los ficheros se leen y se generan en la carpeta `resources`.

---

0. Arquitectura

Sigue la arquitectura **controlador → servicio → repositorio → modelo** de los ejercicios anteriores. Como mínimo:

```
controller/   TiendaController
service/      ProductoService, ClienteService, PedidoService, InformeService
repository/   ProductoRepository (CSV), ClienteRepository (JSON), PedidoRepository (TXT)
model/        Producto, Cliente, Direccion, Pedido, LineaPedido, EstadoPedido (enum)
exception/    DatosInvalidosException, EntidadNoEncontradaException, StockInsuficienteException
```

Reglas:

- **Solo los repositorios** acceden a los ficheros de datos. Los servicios no abren ficheros, salvo `InformeService`, que escribe ficheros de salida.
- **Los servicios** contienen la lógica y las validaciones, y lanzan las excepciones personalizadas.
- **El controlador** captura las excepciones y muestra mensajes por consola. No debe aparecer ningún stack trace al usuario.

---

1. Modelo

Producto

`id`, `nombre`, `categoria`, `precio` (double), `stock` (int) y `etiquetas` (`List<String>`). En el CSV las etiquetas van separadas por `|` y el campo puede venir vacío.

Cliente

`id`, `nombre`, `apellidos`, `email`, `fechaAlta` (**`LocalDate`**; en el JSON viene como `dd/MM/yyyy`), `direccion` (objeto `Direccion` con `calle`, `ciudad` y `cp`, **puede ser null**), `telefonos` (`List<String>`) y `vip` (boolean).

Pedido

`id`, `fecha` (`LocalDate`), `cliente` (**referencia al objeto `Cliente`**, no solo su id), `lineas` (`List<LineaPedido>`) y `estado` (`EstadoPedido`).

LineaPedido

`producto` (referencia al objeto `Producto`), `cantidad` y `precioUnitario` (el precio en el momento de la compra).

EstadoPedido (enum)

`PENDIENTE`, `ENVIADO`, `ENTREGADO`, `CANCELADO`.

---

2. Log de errores con logger

Todos los errores se registran con un **logger** (`logger.error(...)`), **no** con `System.out` ni escribiendo el fichero a mano. Usa **SLF4J + Logback**:

- Cada clase tiene su propio logger:

```
        private static final Logger logger = LoggerFactory.getLogger(ProductoRepository.class);
```

- Usa los niveles `logger.error`, `logger.warn` y `logger.info`, y los parámetros `{}` en lugar de concatenar:

```
        logger.error("Producto {} descartado: {}", id, motivo);
```

- Si hay una excepción inesperada, pásala como último argumento para que la traza quede en el log: `logger.error("Error inesperado", e);`.
- Configura `src/main/resources/logback.xml` para que todo se **añada** (`append`) a `resources/tienda.log` con este formato:

```
[2026-11-08 16:30:12] [ERROR] [ProductoRepository] Producto P005 (línea 6) descartado: falta el nombre
[2026-11-08 16:30:12] [WARN ] [PedidoService] Stock bajo del producto P004 (quedan 1 unidades)
[2026-11-08 16:30:15] [INFO ] [ClienteService] Cliente C013 dado de alta correctamente
```

Patrón de Logback: `[%d{yyyy-MM-dd HH:mm:ss}] [%-5level] [%logger{0}] %msg%n`

---

3. Carga y validación

El orden de carga importa: **productos → clientes → pedidos**, porque los pedidos referencian a los otros dos.

3.1 Productos (CSV)

Un producto **no se carga** (se registra un `ERROR` en el log indicando el motivo) si:

- No tiene `id`, `nombre` o `categoria`.
- El `precio` no es un número o es ≤ 0.
- El `stock` no es un entero o es negativo.
- El `id` ya existe (**duplicado**): se conserva el primero que aparece.
- La línea no tiene exactamente el número de columnas de la cabecera.

Una línea errónea **no debe parar la carga** del resto. Registra en el log también el **número de línea**.

3.2 Clientes (JSON)

¡Ojo! El fichero contiene objetos con errores. Un objeto erróneo **solo debe descartar ese cliente**: el resto se tiene que cargar igualmente. Tienes que contemplar tres tipos de error.

a) Errores de sintaxis dentro de un objeto (falta una coma o una comilla, por ejemplo C007). Si parseas el fichero entero de una vez, Gson lanza una excepción y **pierdes todos los clientes**. Debes separar el fichero en objetos `{...}` y parsear cada uno por separado.

b) Errores de tipo: el campo existe pero tiene un tipo que no toca.

- `"direccion": "Calle Sol 3, Valencia"` → debe ser un objeto o `null`.
- `"telefonos": "699001122"` → debe ser una lista.
- `"vip": "si"` → debe ser `true` o `false`.
- Un campo de texto (`nombre`, `email`...) que venga como objeto o lista.

c) Errores de contenido:

- Falta `id`, `nombre` o `email`.
- El `email` no tiene un formato válido. Usa una expresión regular sencilla, por ejemplo `^[\w.-]+@[\w-]+\.[a-z]{2,}$`.
- La `fecha_alta` no se puede parsear.
- Algún teléfono no tiene 9 dígitos.
- La dirección no tiene `calle` o `ciudad`.

`direccion` a `null` y `telefonos` vacío **son válidos**.

En el log indica **el id del cliente (si se puede leer) y la línea** donde empieza el objeto.

3.3 Pedidos (TXT)

Cada línea tiene el formato `idPedido;fecha;idCliente;lineas;estado`, y las líneas de producto tienen la forma `P001x2,P003x1`.

- Ignora las líneas vacías y las que empiezan por `#`.
- Una línea sin los 5 campos, o cuyo id no tenga la forma `PED-número`, es errónea.
- Un pedido no se carga si la fecha no es válida (`setLenient(false)` o `DateTimeFormatter` estricto), el cliente no existe, **algún** producto no existe, una cantidad no es un entero positivo o el estado no pertenece al enum.
- Cargar el histórico **no modifica el stock**: el stock del CSV ya es el actual.
- En las líneas cargadas, usa como `precioUnitario` el precio actual del producto.

Prueba desde el controlador a listar productos, clientes y pedidos.

---

4. Gestión de productos

1. **Listar** todos los productos ordenados por categoría y, dentro de cada categoría, por precio.
2. **Buscar** por id y **buscar por etiqueta** (devuelve todos los productos que tengan esa etiqueta).
3. **Añadir** un producto con las mismas validaciones de la carga. Si no es válido, lanza `DatosInvalidosException` y regístralo en el log.
4. **Eliminar** un producto por id. **No** se puede eliminar si aparece en algún pedido `PENDIENTE` o `ENVIADO`. Informa del motivo.
5. **Reponer stock**: suma unidades a un producto.

Después de cada operación que modifique datos, **persiste los cambios en** `productos.csv`, con la cabecera y el mismo formato que el original.

---

5. Gestión de clientes

1. **Añadir** un cliente. El id se genera automáticamente a partir del **mayor id que aparezca en el fichero, incluidos los clientes descartados**, para no repetir un id que ya existe en el fichero aunque sea erróneo. Con los datos de partida, el primero será `C013`.
2. **Añadir un teléfono** a un cliente. No se permiten duplicados y el teléfono debe tener 9 dígitos.
3. **Hacer VIP** a un cliente.
4. Persiste los cambios en `clientes.json`, respetando la estructura anidada y escribiendo la fecha otra vez como `dd/MM/yyyy`.

---

6. Gestión de pedidos

6.1 Crear pedido

El controlador recibe un id de cliente y un mapa `idProducto → cantidad`. El servicio debe:

- Comprobar que el cliente y todos los productos existen.
- Comprobar que hay stock suficiente de **todos** los productos **antes** de descontar nada. Si falta stock de uno, no se descuenta de ninguno (lanza `StockInsuficienteException`).
- Descontar el stock, generar el id (`PED-011`, ..., siguiente al mayor del fichero, incluidos los descartados), poner la fecha de hoy y el estado `PENDIENTE`.
- Si algún producto queda con menos de 5 unidades, registrar un `WARN` en el log.
- **Añadir el pedido al final de** `pedidos.txt` (append) y guardar `productos.csv`.

6.2 Cambiar estado

Solo se permiten estas transiciones:

```
PENDIENTE → ENVIADO → ENTREGADO
PENDIENTE → CANCELADO
```

Cualquier otra transición se rechaza con un mensaje. Si un pedido pasa a `CANCELADO`, **el stock se devuelve** a los productos. Como el TXT tiene un pedido por línea, al cambiar un estado tendrás que **reescribir el fichero completo**.

6.3 Factura

Dado el id de un pedido, genera `resources/facturas/factura_PED-XXX.txt` (crea la carpeta si no existe):

```
========================================
             TIENDATECH
========================================
Factura:   PED-008
Fecha:     12/10/2026
Cliente:   Pablo Gómez Torres (C002)
Dirección: Av. de América 45, 28002 Madrid
----------------------------------------
Producto              Cant.  P.Unit   Total
Teclado mecánico          2   79.90  159.80
Altavoz portátil          1   45.00   45.00
Cable HDMI 2m             1    8.99    8.99
----------------------------------------
Subtotal:                          213.79
Descuento VIP (10%):                 0.00
Base imponible:                    213.79
IVA (21%):                          44.90
TOTAL:                             258.69
========================================
```

- Los clientes **VIP** tienen un **10 % de descuento** sobre el subtotal, que se aplica antes del IVA.
- Si el cliente no tiene dirección, escribe `Dirección: no disponible`.
- Usa `String.format` para alinear las columnas y redondea a 2 decimales.
- No se puede generar la factura de un pedido `CANCELADO`.

---

7. Informes

Solo cuentan como **venta** los pedidos `ENVIADO` y `ENTREGADO`.

1. **`ventas_por_categoria.csv`**, con la cabecera `categoria,unidades_vendidas,importe_total`, ordenado por importe de mayor a menor. El importe es el bruto: cantidad × precio unitario, sin descuento ni IVA.

2. **`informe_clientes.json`**: un array con un objeto por cliente con esta estructura:

```
        {
          "id": "C001",
          "nombre_completo": "Lucía Martín Ruiz",
          "vip": true,
          "numero_pedidos": 1,
          "total_gastado": 140.37,
          "producto_favorito": "Ratón inalámbrico"
        }
```

`total_gastado` es la suma de los **totales con descuento e IVA** de sus ventas. `producto_favorito` es el producto del que más unidades ha comprado, o `null` si no ha comprado nada.

3. **`stock_bajo.txt`**: el listado legible de los productos con stock menor que un umbral que recibe el método, ordenado de menos a más stock.
4. **Top N** de los productos más vendidos por unidades, mostrado por consola.
5. **`pedidos_<CIUDAD>.csv`**: dada una ciudad, todos los pedidos de clientes de esa ciudad, con la cabecera `id_pedido,fecha,cliente,estado,total`.

---

8. Copias de seguridad

Antes de **sobrescribir** cualquier fichero de datos (`productos.csv`, `clientes.json` o `pedidos.txt`), copia la versión anterior a:

```
resources/backup/<nombre>_<yyyyMMdd_HHmmss>.<ext>
```

Por ejemplo, `resources/backup/productos_20261108_163012.csv`. Además, implementa un método que **restaure** un fichero a partir de la copia más reciente.

---

9. Segunda versión (ampliación)

Nos piden guardar el **proveedor** de cada producto, que tiene `nombre`, `pais` y `email`. El proveedor puede ser **null**.

Como el CSV no admite objetos anidados:

1. Añade las columnas `proveedor_nombre,proveedor_pais,proveedor_email` al CSV. Deja los campos vacíos si no hay proveedor.
2. Crea un método de **migración** que lea el `productos.csv` antiguo y genere `productos_v2.csv` con las nuevas columnas vacías, sin perder datos.
3. Adapta la lectura para que funcione **con los dos formatos**. Detecta la versión por la cabecera.
4. Genera `productos_<PAIS>.json`: dado un país, todos los productos de proveedores de ese país, con el proveedor anidado como objeto.

---

10. Extras opcionales

- Un **menú por consola** (`Scanner`) para usar todas las funcionalidades.
- **Tests JUnit** para las validaciones y para el cálculo de la factura.
- Un fichero `config.properties` con el IVA, el descuento VIP y el umbral de stock bajo, en lugar de tener esos valores escritos en el código.

Resultados esperados (para que te autocorrijas)

Con los ficheros de partida:

Productos: se cargan **8** (P001, P002, P003, P004, P006, P009, P010 y P011). Se descartan P005 (sin nombre), P007 (precio negativo), P008 (stock no numérico), el segundo P002 (duplicado) y P013 (le faltan columnas).

Clientes: se cargan **5** (C001, C002, C004, C006 y C012).

Cliente	Motivo del descarte
C003	sin email
C005	email inválido
C007	JSON mal formado (falta una coma)
C008	`direccion` es un texto, no un objeto
C009	`telefonos` es un texto, no una lista
C010	teléfono de 5 dígitos
C011	`vip` vale `"si"`

C012 está **después** de los objetos rotos: si no se carga, tu lectura no es tolerante a errores.

Pedidos: se cargan **6** (PED-001, 002, 003, 007, 008 y 009). Se descartan PED-004 (el cliente C099 no existe), PED-005 (la fecha 32/10 no es válida), PED-006 (el producto P020 no existe), la línea 14 (le falta el id) y PED-010 (el estado DEVUELTO no es válido).

Log: tras la carga deben aparecer **17 líneas** `[ERROR]` (5 de productos, 7 de clientes y 5 de pedidos).

Ventas por categoría (pedidos 001, 002 y 008):

categoria	unidades	importe
Portátiles	1	1099.99
Periféricos	5	288.70
Audio	1	45.00
Cables	1	8.99

Factura PED-001 (Lucía es VIP): subtotal 128.90 → descuento 12.89 → base 116.01 → IVA 24.36 → **total 140.37**.

Factura PED-008: **total 258.69** (ver el ejemplo del apartado 6.3).

Top 3 más vendidos: Teclado mecánico (3), Ratón inalámbrico (2) y, empatados con 1 unidad, Portátil Ultra 14, Altavoz portátil y Cable HDMI 2m.

Pruebas de error recomendadas desde el controlador:

- Crear un pedido de 1 unidad de P011 (stock 0) → `StockInsuficienteException`.
- Eliminar P001 → se elimina (solo está en pedidos ENTREGADO).
- Eliminar P010 → rechazado (está en PED-003, que está PENDIENTE).
- Pasar PED-001 (ENTREGADO) a PENDIENTE → transición no permitida.
- Cancelar PED-003 → el stock de P010 vuelve a 42 y el de P006 a 203.
- Generar la factura de PED-007 → rechazado (el pedido está CANCELADO).